

BASH SCRIPTING FUNDAMENTALS

Errors & Debugging

Exit codes, strict mode, traps, and tracing

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Exit codes and status basics — 7 topics
- 2 Section 2: Status in control flow — 7 topics
- 3 Section 3: Strict mode — 7 topics
- 4 Section 4: Intentional failures — 7 topics
- 5 Section 5: Arguments and preflight checks — 7 topics
- 6 Section 6: Traps and cleanup — 7 topics
- 7 Section 7: Logging and runtime switches — 7 topics
- 8 Section 8: Tracing with xtrace — 7 topics

9 Section 9: Syntax checks and ShellCheck — 7 topics

1 Section 10: Small tests and checkpoints — 7 topics

0

1 Section 11: Resilience patterns — 7 topics

1

1 Section 12: Production mistakes and final checks — 7 topics

2

1 Section 1

Exit codes and status basics

1.1 Exit codes are program results

- Every command finishes with an integer status

Successful command

```
pwd  
echo $?
```

- Zero means success by long Unix convention

Failing command

```
ls /missing  
echo $?
```

1.2 Exit 0 means success

- Use 0 when the script completed its contract

Explicit success

```
echo done  
exit 0
```

- A successful script should be quiet or useful

Success in if

```
if ./build.sh; then echo ok; fi
```

1.3 Exit 1 means general failure

- Use 1 for ordinary runtime failures

Report failure

```
echo 'missing input' >&2  
exit 1
```

- Keep the error message specific

Caller sees failure

```
./job.sh || echo failed
```

1.4 Exit 2 means bad usage

- Use 2 when arguments or options are invalid

Usage error

```
echo 'usage: tool FILE' >&2  
exit 2
```

- Print usage before exiting

Too few args

```
[ "$#" -eq 1 ] || exit 2
```


1.5 Exit 126 means not executable

- The command was found but could not run

Permission failure

```
touch script.sh  
./script.sh
```

- Permissions are the common cause

Make executable

```
chmod +x script.sh  
./script.sh
```

1.6 Exit 127 means command not found

- The shell could not locate the command name

Missing command

```
not_a_command  
echo $?
```

- Check spelling and PATH

Check PATH

```
command -v tar || echo missing
```

1.7 \$? must be read immediately

- The status variable changes after every command

Capture status

```
grep ERROR app.log  
status=$?
```

- Even echo overwrites the previous status

Overwritten status

```
grep ERROR app.log  
echo checked  
echo $?
```

2 Section 2

Status in control flow

2.1 true always succeeds

- true is useful for tests and placeholders

Status 0

```
true  
echo $?
```

- It returns status 0 every time

Intentional loop

```
while true; do date; break; done
```

2.2 false always fails

- false returns status 1 every time

Status 1

```
false  
echo $?
```

- It is useful for exercising failure paths

Failure branch

```
if false; then echo yes; else echo no; fi
```

2.3 if reads command status

- Bash tests success and failure directly

Directory exists

```
if cd /tmp; then pwd; fi
```

- No separate boolean type is required

Command as condition

```
if grep -q root /etc/passwd; then echo found; fi
```

2.4 && runs after success

- The right command runs only if the left succeeds

Dependent command

```
mkdir -p out && echo ready
```

- Use it for simple dependent steps

Stop chain

```
false && echo never
```


2.5 || runs after failure

- The right command runs only if the left fails

Fallback

```
cd project || exit 1
```

- Use it for fallback or error handling

Message on failure

```
cp a b || echo copy failed
```

2.6 Status propagation matters

- Functions and scripts return the last command status

Hidden failure

```
check() { false; echo done; }  
check; echo $?
```

- An accidental final echo can hide failure

Preserved failure

```
check() { false; return $?; }  
check
```

2.7 Use command `|| die`

- A die helper keeps fatal failures consistent

Define die

```
die() { echo "$*" >&2; exit 1; }
```

- The failing command stays next to its message

Use die

```
cp "$src" "$dst" || die 'copy failed'
```

3 Section 3

Strict mode



3.1 set -e exits on many failures

- errexit stops the script after many failed commands

Enable errexit

```
set -e  
cp missing out
```

- It reduces accidental continuation

Local enable

```
set -e  
build_project
```

3.2 errexit is ignored in if tests

- A failing condition is expected control flow

If condition

```
set -e  
if grep -q x file; then echo found; fi
```

- set -e does not exit inside the if test

Else branch

```
if cd app; then pwd; else echo no; fi
```

3.3 errexit has && and || exceptions

- Failures before && or || are part of branching

Allowed failure

```
set -e  
false || echo handled
```

- Bash does not exit for those expected statuses

Success chain

```
set -e  
mkdir -p out && touch out/a
```

3.4 set -u catches unset variables

- nounset fails when an unset variable is expanded

Enable nounset

```
set -u  
echo "$missing"
```

- It catches typos early

Safe default

```
echo "${name:-guest}"
```


3.5 nounset pitfalls are predictable

- Optional variables need default syntax

Require value

```
echo "${API_TOKEN:?missing token}"
```

- Empty and unset are not the same idea

Optional first arg

```
first="${1:-}"  
echo "$first"
```

3.6 pipefail exposes pipeline failures

- Without pipefail only the last command decides

Masked failure

```
grep x missing | sort  
echo $?
```

- Earlier pipeline failures can be masked

Enable pipefail

```
set -o pipefail  
grep x missing | sort
```

3.7 Use set -euo pipefail together

- The trio catches failed commands and missing values

Strict header

```
set -euo pipefail
```

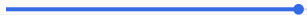
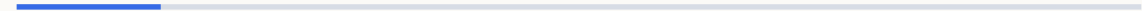
- It is a strong default for new scripts

Handled exception

```
grep -q x file || true
```

4 Section 4

Intentional failures



4.1 Use || true only intentionally

- Some commands fail for acceptable reasons

Optional cleanup

```
rm -f old.tmp || true
```

- Document why the failure is harmless

Expected no match

```
grep -q TODO file || true
```

4.2 grep no match is status 1

- grep uses 1 for no matching lines

No match allowed

```
if grep -q ERROR log; then echo bad; fi
```

- No match is often data, not a crash

Count with fallback

```
grep -c ERROR log || true
```

4.3 Fail early for required steps

- Stop when later steps depend on earlier success

Required copy

```
cp config.yml out/config.yml
```

- Early failure keeps damage small

Required directory

```
cd app || exit 1
```

4.4 Accumulate nonfatal errors

- Some scripts should process everything possible

Count failures

```
errors=0  
cp a b || errors=$((errors+1))
```

- Track failures and report them at the end

Final status

```
[ "$errors" -eq 0 ] || exit 1
```


4.5 return exits a function

- return sets the function status

Return failure

```
check() { [ -f "$1" ] || return 1; }
```

- It does not exit the whole script

Call check

```
check config.yml || echo missing
```

4.6 exit ends the shell script

- exit terminates the current shell process

Fatal error

```
[ -f config.yml ] || exit 1
```

- Use it for fatal top-level errors

Explicit code

```
exit 2
```

4.7 Strict mode in sourced files needs care

- A sourced file changes the caller shell

Sourcing changes shell

```
. ./lib.sh
```

- set options can leak to the importing script

Library function

```
load_config() { [ -f "$1" ] || return 1; }
```

5 Section 5

Arguments and preflight checks

5.1 Check the number of arguments

- \$# contains the positional argument count

Require one arg

```
[ "$#" -eq 1 ] || exit 2
```

- Validate before using \$1 or \$2

Require two args

```
[ "$#" -eq 2 ] || { echo usage >&2; exit 2; }
```

5.2 Put usage in a function

- A usage function keeps help text consistent

Usage function

```
usage() { echo 'usage: app FILE' >&2; }
```

- Write usage to stderr for errors

Use it

```
[ "$#" -eq 1 ] || { usage; exit 2; }
```

5.3 Validate option values

- Options can be present but invalid

Mode check

```
[ "$mode" = fast ] || [ "$mode" = safe ]
```

- Check allowed values near parsing

Reject value

```
echo 'invalid mode' >&2  
exit 2
```

5.4 Require important environment variables

- Environment variables are inputs too

Require env

```
: "${API_TOKEN:?API_TOKEN required}"
```

- Use parameter expansion for clear errors

Default env

```
LOG_LEVEL="${LOG_LEVEL:-info}"
```


5.5 need checks dependencies

- Check external commands before the main work

Define need

```
need() { command -v "$1" >/dev/null || exit 127; }
```

- command -v is portable and quiet

Check tools

```
need tar  
need gzip
```

5.6 Assert a file exists

- Use file tests for required inputs

File assertion

```
[ -f "$file" ] || { echo "missing: $file" >&2; exit 1; }
```

- Quote the path being tested

Directory assertion

```
[ -d "$dir" ] || exit 1
```

5.7 Assert a directory is writable

- Many scripts fail later when output is not writable

Writable check

```
[ -w "$outdir" ] || exit 1
```

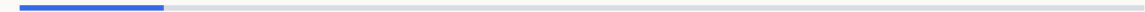
- Test the output directory before generating files

Create first

```
mkdir -p "$outdir"  
[ -w "$outdir" ]
```

6 Section 6

Traps and cleanup



6.1 trap EXIT always runs at script end

- EXIT traps run on success and failure

Exit trap

```
cleanup() { rm -f "$tmp"; }  
trap cleanup EXIT
```

- Use them for cleanup and final messages

Final message

```
trap 'echo done >&2' EXIT
```

6.2 Clean up temp directories

- `mktemp -d` creates a unique workspace

Temp dir

```
tmp=$(mktemp -d)  
trap 'rm -rf "$tmp"' EXIT
```

- An EXIT trap removes it even on failure

Use temp dir

```
cp input "$tmp/input"
```

6.3 trap INT handles Ctrl-C

- INT is sent by an interactive interrupt

Interrupt trap

```
trap 'echo cancelled >&2; exit 130' INT
```

- Use it to print a clear cancellation message

Long task

```
sleep 30
```

6.4 trap ERR handles many failures

- ERR traps run when errexit would react

ERR trap

```
trap 'echo failed at $LINENO >&2' ERR
set -e
```

- They are useful for logging failure context

Trigger

```
false
```


6.5 ERR trap caveats mirror errexit

- ERR is skipped in expected failure positions

No ERR in if

```
set -Eeo pipefail
if false; then echo ok; fi
```

- if tests and || handlers do not trigger it

No ERR when handled

```
set -Eeo pipefail
false || echo handled
```

6.6 errtrace carries ERR into functions

- set -E enables ERR inheritance in functions

Enable errtrace

```
set -Ee
trap 'echo ERR $LINENO >&2' ERR
```

- It is also written as set -o errtrace

Function failure

```
work() { false; }
work
```

6.7 inherit_errexit helps command substitutions

- Command substitutions can weaken errexit behavior

Enable option

```
shopt -s inherit_errexit
```

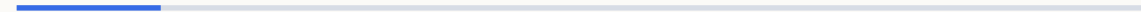
- inherit_errexit keeps failure behavior stricter

Substitution

```
value=$(failing_command)
```

7 Section 7

Logging and runtime switches



7.1 Log errors to stderr

- stdout is for normal script output

Error message

```
echo 'copy failed' >&2
```

- stderr is for diagnostics and errors

Normal output

```
echo "$result"
```

7.2 Add timestamps to logs

- Timestamps make production reports easier to follow

Timestamp

```
date '+%Y-%m-%d %H:%M:%S'
```

- Use a consistent format

Log line

```
echo "$(date '+%F %T') start" >&2
```

7.3 Use log levels

- Levels separate info from warnings and errors

Info

```
echo 'INFO starting' >&2
```

- Simple prefixes help when reading files

Error

```
echo 'ERROR missing file' >&2
```

7.4 Verbose flags reveal progress

- Verbose mode is useful for humans running scripts

Verbose check

```
[ "${verbose:-0}" -eq 1 ] && echo working >&2
```

- Keep default output concise

Enable flag

```
verbose=1  
[ "$verbose" -eq 1 ] && echo step >&2
```


7.5 Debug environment variables are quick toggles

- Environment variables can enable debugging without parsing

Debug toggle

```
[ "${DEBUG:-0}" = 1 ] && set -x
```

- Default them safely under set -u

Run with debug

```
DEBUG=1 ./backup.sh
```

7.6 Dry-run flags prevent changes

- Dry runs show what would happen

Dry-run variable

```
dry_run=1
```

- They are safest when routed through one helper

Guard action

```
[ "$dry_run" = 1 ] && echo rm file || rm file
```

7.7 Echo commands before running risky work

- A run helper makes command logging consistent

Run helper

```
run() { echo "+ $" >&2; "$@"; }
```

- It pairs naturally with dry-run mode

Use helper

```
run mkdir -p out
```

8 Section 8

Tracing with xtrace

8.1 bash -x traces a script

- xtrace prints commands after expansion

Trace script

```
bash -x ./script.sh
```

- It helps reveal what Bash is really running

Trace with args

```
bash -x ./script.sh input.txt
```

8.2 set -x starts tracing inside a script

- Enable tracing around the suspicious area

Start trace

```
set -x  
cp "$src" "$dst"
```

- Tracing everything can be noisy

Trace function

```
set -x  
work  
set +x
```

8.3 set +x stops tracing

- Stop tracing before secrets or noisy loops

Stop trace

```
set +x
```

- Keep traces focused and readable

Around block

```
set -x  
build  
set +x
```

8.4 PS4 customizes trace prefixes

- PS4 controls the prefix before traced commands

Line numbers

```
PS4='+ $LINENO: '  
set -x
```

- Include line numbers for faster debugging

Function names

```
PS4='+ ${FUNCNAME[0]}:$LINENO: '
```


8.5 Debug functions one at a time

- Small functions are easier to trace

Trace call

```
set -x  
make_report "$file"  
set +x
```

- Trace the function call and its inputs

Check status

```
make_report "$file"  
echo $?
```

8.6 Protect secrets while tracing

- xtrace prints expanded values

Stop before secret

```
set +x  
curl -H "Auth: $TOKEN" url
```

- Disable tracing around tokens and passwords

Resume after

```
set -x  
next_step
```

8.7 Make a minimal reproduction

- Copy only the failing lines into a tiny script

Tiny script

```
printf '%s\n'  
a b | grep c
```

- Use fixed sample input

Trace it

```
bash -x repro.sh
```

9 Section 9

Syntax checks and ShellCheck

9.1 bash -n checks syntax

- No commands are executed

Syntax check

```
bash -n script.sh
```

- It catches parse errors before runtime

Check all scripts

```
bash -n scripts/*.sh
```

9.2 ShellCheck finds common bugs

- ShellCheck is a static analyzer for shell scripts

Run ShellCheck

```
shellcheck script.sh
```

- It catches quoting and portability issues

Several files

```
shellcheck bin/*.sh
```

9.3 SC2086 warns about unquoted expansions

- Unquoted variables can split into many words

Risky

```
cp $src $dst
```

- They can also expand globs accidentally

Safer

```
cp "$src" "$dst"
```

9.4 SC2154 warns about unknown variables

- Typos in variable names are common

Typo

```
echo "$ouput"
```

- ShellCheck catches many before runtime

Correct

```
echo "$output"
```


9.5 SC2164 warns about unchecked cd

- A failed cd can make later commands run elsewhere

Unchecked

```
cd "$dir"  
rm -f *.tmp
```

- Check cd or let strict mode stop the script

Checked

```
cd "$dir" || exit 1
```

9.6 SC2046 warns about command splitting

- Unquoted command substitution can split output

Risky

```
rm $(cat files.txt)
```

- Prefer arrays or while read loops

Quoted string

```
name="$(cat name.txt)"
```

9.7 Ignore warnings only with a reason

- Some warnings are false positives for your intent

Inline ignore

```
# shellcheck disable=SC2086  
cmd $flags
```

- Keep ignores narrow

File check

```
shellcheck script.sh
```

10

Section 10

Small tests and checkpoints

10.1 Compare expected output

- A tiny expected file makes behavior concrete

Capture output

```
./report.sh > actual.txt
```

- diff returns nonzero when output differs

Compare

```
diff -u expected.txt actual.txt
```

10.2 Write small test scripts

- Tests can be plain Bash at first

Test file

```
bash tests/report_test.sh
```

- Each test should set up its own inputs

Fail assertion

```
[ -s report.txt ] || exit 1
```

10.3 Use temp fixtures

- Fixtures keep tests repeatable

Fixture dir

```
tmp=$(mktemp -d)  
trap 'rm -rf "$tmp"' EXIT
```

- Temporary directories avoid polluting the project

Fixture file

```
printf 'ERROR x  
' > "$tmp/app.log"
```

10.4 Assert exit codes

- Some tests care more about status than output

Capture status

```
./tool missing  
status=$?
```

- Capture the status immediately

Assert status

```
[ "$status" -eq 2 ] || exit 1
```


10.5 Test error messages

- Helpful errors are part of script behavior

Capture stderr

```
./tool missing 2>err.txt
```

- Capture stderr separately

Check message

```
grep -q usage err.txt
```

10.6 Add checkpoint commands

- Checkpoint commands prove assumptions while debugging

Location check

```
pwd >&2
```

- Use pwd, ls, and printf deliberately

Value check

```
printf 'file=%s  
' "$file" >&2
```

10.7 Run a smoke test in CI

- A smoke test catches completely broken scripts

Syntax smoke

```
bash -n script.sh
```

- Run syntax checks before behavior tests

Behavior smoke

```
./script.sh --help >/dev/null
```

11

Section 11

Resilience patterns

11.1 Retry transient commands

- Network and service commands can fail temporarily

Retry loop

```
for i in 1 2 3; do curl -fsS url && break; sleep 1; done
```

- Retry only operations that are safe to repeat

Attempt count

```
attempts=3
```

11.2 Backoff reduces pressure

- Waiting longer between attempts avoids hammering services

Backoff sleep

```
sleep "$i"
```

- Simple linear backoff is often enough

Retry message

```
echo "retry $i" >&2
```

11.3 timeout prevents hangs

- External commands can hang forever

Limit runtime

```
timeout 10s curl -fsS url
```

- timeout stops a command after a limit

Handle timeout

```
timeout 5s ./job.sh || echo timed out
```

11.4 Capture network status clearly

- curl can fail for HTTP or connection reasons

Curl fail fast

```
curl -fsS "$url" -o out.json
```

- Use flags that make failures visible

Handle curl

```
curl -fsS "$url" || die 'download failed'
```


11.5 pipefail motivation is data safety

- A report can look valid after an earlier pipeline error

Unsafe report

```
grep ERROR missing.log | sort > report.txt
```

- pipefail prevents partial data from seeming successful

Safer report

```
set -o pipefail  
grep ERROR missing.log | sort > report.txt
```

11.6 PIPESTATUS shows each stage

- PIPESTATUS stores statuses for the last pipeline

Inspect pipeline

```
grep x file | sort  
echo "${PIPESTATUS[*]}"
```

- It must be read immediately

Save statuses

```
statuses=("${PIPESTATUS[@]}")
```

11.7 Cleanup on failure protects reruns

- Partial files can confuse the next run

Temp output

```
tmp=$(mktemp)  
trap 'rm -f "$tmp"' EXIT
```

- Write to temp files before moving into place

Atomic finish

```
mv "$tmp" report.txt
```

1 2

Section 12

Production mistakes and final checks

12.1 Missing quotes cause surprise words

- Spaces in paths become multiple arguments

Broken path

```
rm $backup_dir/file
```

- Globs can expand unexpectedly

Quoted path

```
rm "$backup_dir/file"
```

12.2 Pipelines can mask failures

- The final command may succeed while earlier work failed

Masked

```
cat missing | wc -l
```

- Reports and filters are common victims

Exposed

```
set -o pipefail  
cat missing | wc -l
```

12.3 Do not lose the real status

- Logging after a command changes \$?

Save first

```
./job.sh  
status=$?
```

- Save status before printing diagnostics

Exit saved

```
echo "status=$status" >&2  
exit "$status"
```

12.4 Unchecked cd is dangerous

- A failed cd leaves the script in the old directory

Check cd

```
cd "$dir" || exit 1
```

- Destructive commands can hit the wrong place

Subshell

```
( cd "$dir" && rm -f *.tmp )
```


12.5 set -u and arrays need defaults

- Empty arrays and unset arrays behave differently

Safe args

```
args=("${extra_args[@]:-}")
```

- Use safe expansion when an array may be empty

Optional scalar

```
name="${name:-}"
```

12.6 Race-free temp files matter

- Predictable temp names can collide

Bad temp

```
tmp=/tmp/report.txt
```

- mktemp creates safer unique paths

Better temp

```
tmp=$(mktemp)  
trap 'rm -f "$tmp"' EXIT
```

12.7 Use a final debugging checklist

- Run `bash -n` before behavior tests

Checklist commands

```
bash -n script.sh  
shellcheck script.sh
```

- Run ShellCheck before sharing

Failure test

```
./script.sh missing || echo failed
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

07 · Errors

